
UNIT 1 INTRODUCTION TO PYTHON

Structure

- 1.0 Introduction
- 1.1 Objectives
- 1.2 History of Python
- 1.3 Need for Python
- 1.4 Packages for cross-platform application of Python
- 1.5 Getting Started with Python
- 1.6 Program Structure in Python
- 1.7 Running the First Program
- 1.8 Summary
- 1.9 Answer to Check your Progress
- 1.10 References/Further Readings

1.0 INTRODUCTION

Python is an easy-to-learn programming language. It is a powerful programming language and is widely used in Artificial Intelligence, Machine Learning, Neural Networks and many other advanced fields of Computer Science. Ideally, it is designed for rapid prototyping of complex applications. Python interfaces with various operating system calls and libraries and can be extended with C, C++, or Java. Many large companies like NASA, Google, Meta, Netflix, YouTube, BitTorrent, ISRO, etc. use Python to run their projects.

To build a career in IT and software development, programming is a basic requirement, but before you start learning, you should be clear about which programming language to choose. Before learning any programming language, you should figure out which language suits you. To date, if you compare C, C++, Java, and Python programming languages, it may help you understand that Python is relatively more popular and a popular choice in the IT Industry. In this course, you will learn Python programming in detail. In this unit, you will learn how to write basic Python programs and run them. A basic comparison of C and Python programming languages is given for your understanding.

Metrics	Python	C
Introduction	It is a high-level, general-purpose & interpreted programming language.	C is a general-purpose procedural programming language.
Speed	As an interpreted programming language, it runs slower than a compiled programming language (i.e., C).	As a compiled programming language, it runs faster than an interpreted programming language (i.e., Python).
Usage	The number of lines of code written in Python is quite less in comparison to C	C syntax is more complicated than Python.

Declaration of variables	Declaration of Variable type is not required; they are untyped, and a given variable can store different types of values at different instances during the execution of any Python program.	In C, declaration of variable type is a must, and it is done at the time of its creation, and only values of that declared type must be assigned to the variables.
Error Debugging	Error debugging is simple because it executes only one instruction at a time. Compilation and execution are performed simultaneously. Errors are instantly shown, and execution stops at that instruction only.	Because C is a compiler-dependent language, error debugging is difficult; it takes the entire source code, compiles it, and then shows all errors.
Function renaming mechanism	The same function can be used by two different names i.e. it supports the mechanism of function renaming	C does not support function renaming; the same function cannot be used under two different names.
Complexity	The syntax of Python programs is quite simple, easy to read, write and learn	The syntax of a C program is comparatively harder than the syntax of Python.
Memory-management	It supports automatic garbage collection for memory management.	In C, the Programmer has to manage memory explicitly.
Applications	Python is a General-Purpose programming language that can also be used for microcontrollers.	C is generally used for hardware-related applications.
Built-in functions	The library of built-in functions is quite large in Python.	The library of built-in functions is quite limited in C
Implementing Data Structures	Gives ease of implementing data structures with built-in insert and append functions.	Explicit implementation of functions is required for the implementation of data structures
Pointers	Python does not support pointers.	Pointers are frequently used in C.

Python is a high-level, general-purpose, interpreted programming language. It is dynamically typed and garbage-collected and supports multiple programming paradigms, including structured (particularly procedural), object-oriented, and functional programming; because of its rich set of standard libraries, Python is often described as a "batteries included" language.

1.1 OBJECTIVES

After going through this unit, you will be able to:

- Describe the history of Python,
 - Explain the need for Python as a programming language,
 - Describe cross-platform applications,
 - Explain the structure of a Python program, and
 - Write and execute Python code.
-

1.2 HISTORY OF PYTHON

Python was conceived in the late 1980s as a successor to the ABC language. It was created by Guido van Rossum in 1989, and its first release was in 1991. Later, Python 2.0, released in 2000, added features like list comprehensions and a garbage collection system that could collect reference cycles. Python 2 implemented the technical details of the Python Enhancement Proposal (PEP), simplifying the code development complexities of earlier versions. But because of stability issues with Python 2.x, Python 2.7 (the last version in 2.x, released in 2010) was discontinued in 2020. Developers resolved issues with Python 2 in parallel, and in 2008, Python 3.0 was released. It was a major revision of the language; Python 3 was mainly released to fix problems in Python 2. As Python 3 was not completely backwards-compatible, much of the Python 2 code does not run unmodified on Python 3. To make migration to Python 3 easier, some Python 3 features were backported to Python 2.x.

Thus, migrating a project from Python 3 to Python 2.x requires many changes, not only to the project and its applications but also to the libraries that make up the Python environment.

As of June 2026 , Python 3.14.6 was released. You can freely download the latest version of Python from <https://www.python.org/downloads/> .

A global community of programmers develops and maintains C-Python, which is an open-source reference implementation. A non-profit organisation, the Python Software Foundation, manages and directs resources for Python and C-Python development. So, what is C-python? It is the default or reference implementation of the Python programming language, and as the name suggests, C-Python is written in the C language. Some of the implementations which are based on the C-Python runtime core but with extended behaviour or features in some aspects are Stackless Python and MicroPython; Stackless Python relates to C-Python with an emphasis on concurrency using tasklets and channels (used by ds-python for the Nintendo DS), and MicroPython relates to working with microcontrollers.

C-Python compiles the Python source code into intermediate bytecode, which is executed by the C-Python virtual machine. It is to be noted that Python is a dynamic programming language. Still, it is said to be slow, because the default C-Python implementation compiles the Python source code in bytecode, which is slow as compared to machine code(native code).C-Python is distributed with a large standard library written in a mixture of C and Python, C-Python provides the highest level of compatibility with Python packages and C extension modules.

Further, Jython, IronPython, and PyPy are also implementations of Python; they also provide a good level of compatibility with Python packages. These implementations are based on Java, .NET and Python, respectively. We will briefly discuss these implementations, starting with Jython.

Jython is actually the Java platform-based implementation of the Python programming language, so that you can run Python programs on the Java platform. Programs written in Jython use Java classes, not Python modules. The Jython compiler compiles the Jython programs into Java bytecode, which can then be run by the Java virtual machine. Jython lets Python programs use Java class library functions. Compared with CPython, Jython is slower and less compatible with CPython libraries. However, the compatibility in IronPython and PyPy is quite good.

IronPython is the implementation of Python written in C# (Microsoft's .NET framework). As Jython used Java Virtual Machine (JVM), IronPython uses .NET Virtual Machine i.e., the Common Language Runtime. The .NET Framework and Python libraries are used efficiently by the IronPython. Further, because of its Just-in-Time (JIT) compiler and lack of a Global Interpreter Lock, IronPython performs better for Python programs that require threads or multiple cores.

Guido van Rossum (creator of Python) said “If you want your code to run faster, you should probably just use PyPy.” —PyPy is an implementation of the Python programming language written in Python itself. The PyPy Interpreter is written in RPython, which is a subset of Python. The PyPy interpreter uses a JIT compiler to compile source code into native machine code, making it very fast and efficient. PyPy also supports stackless mode by default, providing micro-threads for massive concurrency.

1.3 NEED FOR PYTHON

Python is a high-level, interpreted, and dynamically typed programming language. Python requires fewer lines of code compared to languages like Java or C++. This encourages the rapid development of application prototypes, makes debugging of errors easy and identifies itself as the language to code with. As Python is an interpreted language, Python code is executed line by line, allowing you to test changes instantly without waiting for compilation.

Now, you might be in a position to appreciate the simplicity and capability of Python as a programming language. It is a general-purpose language and applies to almost every domain of software development, whether web development, scientific applications, business software, or anything else you can think of. It has wider coverage of applications, and is widely used as it complies with the current needs of the software industry. Thus, it offers a promising future for learners.

It is easy to understand the need of Python if you understand the application areas of Python. You might not know that various applications like YouTube, BitTorrent, DropBox etc. uses Python to achieve their functionality. The reason behind the choice of python, for these applications is the ability of Python to be compatible with cross-platform operating systems (a cross-platform application may run on any operating system, be it Microsoft Windows or Linux, or macOS or any other), which simplifies the development of applications.

Now let us discuss the various types of applications that can be developed using Python; many applications can be built, and a few are given below.

- **Artificial Intelligence(AI) and Machine Learning(ML):** These are the need of the times, as they yield the most promising careers for the learners. AI and ML involve incorporating intelligence into machines through self-learning from data stored in the system; various algorithms support this. But to bring AI and ML into action, Python is the first choice. Why? Because various well-equipped libraries like Pandas, Scikit-Learn, NumPy, etc. are available to facilitate engineers and scientists. You just learn the algorithms, use the library, and you have your solution to the problem. It is that simple.
- **Data Science and Data Visualization:** Data Science is just another promising career; if you know how to extract relevant information from the data source, then the world is yours. But it's not easy; you need various algorithms and techniques to solve the problem. To simplify your work, Python again emerges to help you study your data, perform operations, and extract the required information. Libraries such as Panda and NumPy help you extract information. Further, libraries such as Matplotlib and Seaborn are used for data visualization. These libraries are quite helpful in plotting and visualization of data. This is how Python helps you to become a Data Scientist.
- **Mobile Applications:** In general, people think that Android and iOS are for mobile development, but what about using Python for mobile app development? Historically, Python did not have a strong track record for mobile GUI development, which was questionable since Python is positioned as a solution for a variety of applications. But over time, Python has evolved, and now it is one of the options for Mobile App development. Two well-known frameworks, i.e. Kivy and BeeWare, are options for mobile application development with Python.
- **Embedded Applications and IoT:** We learned from earlier sections of this unit that Python is based on C. Thus, it can be used to create software for embedded applications. MicroPython is a software implementation of Python 3, written in C. In fact, MicroPython is a Python compiler that runs on microcontroller hardware; it includes modules to provide access to low-level hardware. One renowned embedded application is the Raspberry Pi, which uses Python for computing. It can serve as a computer or a simple embedded board for high-level computations; you can use it for IoT or Mobile applications to build smart gadgets. Nowadays, Python is widely used in embedded and IoT application development across microcontrollers, Single-Board Computers (SBCs), and testing environments because of its rapid prototyping capabilities and massive ecosystem.
- **Web Development:** In recent years, Python has dominated web development due to its rich ecosystem of full-stack and micro-frameworks like Django, Flask, and Pyramid, which eliminate boilerplate code by providing built-in backend logic for routing, database management, and robust security middleware. These frameworks seamlessly handle low-level internet protocols—such as managing secure HTTPS and SSL encryption layers or automating file exchanges via FTP—while leveraging Python's native libraries to dynamically parse and serialize core web data formats like JSON, Email and XML etc.

- **Game Development:** Python is rapidly expanding its footprint in the booming gaming industry as a powerful language for building interactive 2D and 3D games through specialized libraries like Pygame and Pyglet for 2D logic, and PySoy for 3D engine layouts. While high-performance commercial engines rely heavily on C++ for core rendering, Python serves as the foundational scripting engine for AI, game loops, and modding tools in major industry titles, including Civilization IV, Disney's Toontown Online, and Vega Strike etc.
- **Enterprise & Business Applications:** Python is a preferred language for building custom internal enterprise software, such as Enterprise Resource Planning (ERP) tools, Customer Relationship Management (CRM) databases, and billing systems, due to its readability and security features.
- **Python for Network Security:** Python is widely used by both offensive penetration testers and defensive security analysts to automate threat mitigation and audit network infrastructure. Using Python and advanced packet-manipulation libraries like Scapy, programmers and developers can craft, sniff, and dissect traffic down to individual protocol bits to identify open ports, forge network packets, and probe vulnerabilities.

So, you can now judge that Python has a variety of applications. No matter what field you take up, Python is rewarding. The above discussion might have helped you understand some application areas of the Python programming language.

Check Your Progress -1

1. Compare Python and C programming languages

.....
.....
.....
.....

2. Discuss different variants of Python viz. C-python, Jython, IronPython, PyPy

.....
.....
.....
.....

3. List some areas of application of the Python programming language.

.....
.....
.....
.....

1.4 PACKAGES FOR CROSS PLATFORM APPLICATION OF PYTHON

Currently, Python is one of the most popular programming languages, and it has marked its presence by contributing actively to every emerging field in computer science. It has a vast set of libraries for almost every field, such as Machine

Learning (Numpy, Pandas, Matplotlib), Artificial Intelligence (PyTorch, TensorFlow), and Game development (Pygame, Pyglet), and many more.

After going through section 1.2 of this unit, you understood the meaning of Cross-platform applications, and you are now aware of the potential of Python as a cross-platform programming language. You also came to know that the development of different types of applications requires different types of packages, libraries, modules, frameworks etc. You might be wondering: what is the difference between these terms? Are they the same or different? Let us clear up this confusion first, and then we will briefly discuss the functionality of different packages used to develop various Python applications.

Python: Framework Vs Library Vs Package Vs Module

A framework is a collection of libraries. It defines the program's architecture.

A library is a collection of packages. (*We may understand that a Python library or framework is a pre-written program that is ready to use on common coding tasks.*)

A package is a collection of modules. It must contain an `__init__.py` file as a flag so that the python interpreter processes it as such. The `__init__.py` could be an empty file without causing issues. A package, in essence, is a directory that holds subpackages and modules. While we can create our own packages, we can also use one from the Python Package Index (PyPI) to use for our projects. To import a package, we **type `import Game.Sound.load`** Or we can also import it, giving it an alias **:`import Game.Sound.load as load game`**.

A module is a file which contains various Python functions and global variables. It is simply a **.py file** that contains executable Python code. We put similar code together in one module. This helps us modularise our code and makes it much easier to manage. A module also provides reusability. With a module, we do not need to rewrite the same code for a new project.

Now let us see how a module and a package differ:

1. A module is a file containing Python code. A package, however, is like a directory that holds sub-packages and modules.
2. A package must hold the file `__init__.py`. This does not apply to modules.
3. To import everything from a module, we use the wildcard `*`. But this does not work with packages.

We will learn more about them, as we proceed in this course. To start, we will discuss various Frameworks and Libraries first, and then you will learn how to use them, including methods and packages. Python has many popular libraries and frameworks. Some of these libraries/frameworks, such as NumPy, Pandas, Kivy, and Matplotlib, you will learn in detail in this course.

Some Popular Python Frameworks/Libraries:

NumPy: A library for scientific computing, also known as Numerical Python. Almost all Python machine-learning tools like Matplotlib, SciPy, Scikit-learn, etc. rely on this library to a reasonable extent. It comes with functions for dealing with complex mathematical operations like linear algebra, Fourier transformation, random numbers, along with features that work with matrices and n-arrays in Python. It is widely used in handling sound waves, images, and other binary functions.

Pandas: It is an open-source Python library that offers a wide range of tools for data analysis and manipulation. With Pandas, data from a broad range of sources like CSV, SQL databases, JSON files, and Excel can be read. Data analysis and manipulation is a prerequisite for most of the machine learning projects, where a significant amount of time is spent analysing the trends and patterns hidden in the datasets. Using Pandas, one can manage complex data operations with just one or two commands; it comes with several inbuilt methods for data handling, and it also serves as the starting point to create more focused and powerful data tools.

Kivy: It is an open-source Python Framework used for cross-platform Python GUI mobile app development. It lets you write pure-Python graphical applications that run on desktop platforms (Windows, Linux, macOS, and Raspberry Pi) and on iOS & Android.

Django: A web framework built in Python. It encourages rapid web application development. Developers can focus on writing the app only. Its primary goal is to ease the creation of complex, database-driven websites. Django emphasises code reusability and pluggability of components; thus, it supports less code, low coupling, and helps in rapid website development .

Matplotlib : This library is specifically meant for Data Science; it helps to generate data visualization like 2D diagrams and graphs like histograms, scatter plots, and even graphs for non-cartesian coordinates. It provides an object-oriented API that lets you embed plots directly into applications. This library helps Python compete with scientific tools like MATLAB and Mathematica. It is quite compatible with the popular plotting libraries, but developers are required to write more code than usual, while using this library for generating advanced visualizations.

TensorFlow: TensorFlow is an end-to-end Python machine learning library for performing high-end numerical computations. TensorFlow can handle Deep Neural Networks for image recognition, handwritten digit classification, recurrent neural networks, and NLP (Natural Language Processing).

Keras: A leading open-source Python library for developing neural networks and Machine Learning projects. It simplifies designing and developing Neural Networks for beginners in Machine Learning. Keras also supports Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), which are essential in deep learning. It includes algorithms for normalization, optimization, and activation layers. Rather than being an end-to-end Python machine learning library, Keras is a user-friendly, extensible interface that enhances modularity and expressiveness.

Theano : It is a library for scientific computation; it allows you to define, optimize as well as evaluate complex mathematical expressions, which deal with multidimensional arrays. Repeated computation of complex mathematical expressions underpins many AI and ML applications. Theano performs data-intensive calculations quickly; it runs almost 100 times faster than on a CPU alone. It aims to reduce the development and execution time of ML apps, particularly in deep learning algorithms.

Scikit-learn: A prominent open-source machine learning library in Python. It includes a wide range of algorithms like DBSCAN, gradient boosting, random forests, support vector machines, and k-means, which are generally used to implement various machine learning concepts, i.e., Classification, Clustering, Regression, etc. It can interoperate with numeric and scientific libraries of

Python like NumPy and SciPy. Scikit-learn supports both supervised and unsupervised ML.

SciPy: It is a library for all types of scientific programming projects. Its main functionality is built upon NumPy, and it includes modules for linear algebra, Integration, optimization, and statistics too. SciPy is supported with extensive documentation, which makes its working really easy.

Seaborn : It is an unparalleled visualization library, based on Matplotlib's foundations. Seaborn offers a high-level dataset-based interface to make amazing statistical graphics. With Seaborn, it is simple to create certain types of plots like time series, heat maps, and violin plots. The functionalities of Seaborn go beyond Pandas and matplotlib, with features to perform statistical estimation when combining data across observations, and to plot and visualise the suitability of statistical models to strengthen dataset patterns.

Scrapy : It is the most widely used Python library for data science, specifically data mining. It is used to build crawling programs i.e spider bots, which are used to retrieve structured data (example URLs or Contact Information) from the web. Developers use it for gathering data from APIs.

Pygame : It is a python programming language library for making multimedia applications like video games, animations etc. It includes libraries for computer graphics and sound, designed to be used with Python. Pygame is suitable to create client-side applications that can be potentially wrapped in a standalone executable.

MicroPython : It is a software implementation of Python 3, written in C. In fact, MicroPython is a Python compiler that runs on the hardware of micro-controller, it includes modules to provide access to low-level hardware. One renowned embedded application is the Raspberry Pi, which uses Python for computing. It can be used as a computer or as a simple embedded board to perform high-level computations; you can use it for IoT or Mobile applications, thus producing your smart gadgets.

BeeWare: A Python GUI and mobile development framework for building native Python Mobile Apps. The "BeeWare" project provides a set of tools and an abstraction layer, which can be used to write native-looking mobile and desktop applications using Python.

1.5 GETTING STARTED WITH PYTHON

Now, we have some clarity about Python as a programming language. From the previous sections, we know about Python's various libraries and frameworks. Now, we need to work on the IDEs (Integrated Development Environments) of Python; there are many IDEs like Jupyter Notebook, Spyder, VS Code, R Programming, etc. All these are collectively available in Anaconda (Anaconda is a free and open-source distribution of the Python and R-Programming languages for scientific computing (data science, machine learning applications, large-scale data processing, predictive analytics, etc.)), or you may also go for the cloud versions like Google Colab Notebook, where you do not need high-configuration hardware; only internet is required, and through your Gmail account you can work on Python using Jupyter Notebook.

Here we will briefly discuss some ways to work with Python; you may choose any or try all, and also explore other options.

1. Just browse for <https://www.python.org> and perform the following steps:

- Download the latest version of Python for the operating system installed on your computer; in this case, it's Windows 64-bit, so I have downloaded python-3.7.7 (python-3.7.7-amd64.exe) from <https://www.python.org/downloads/release/python-377/>
- Now Run this. exe file and install Python; just click Next and go ahead till the setup installation is finished
- Finally, you will have an interface for Python programming

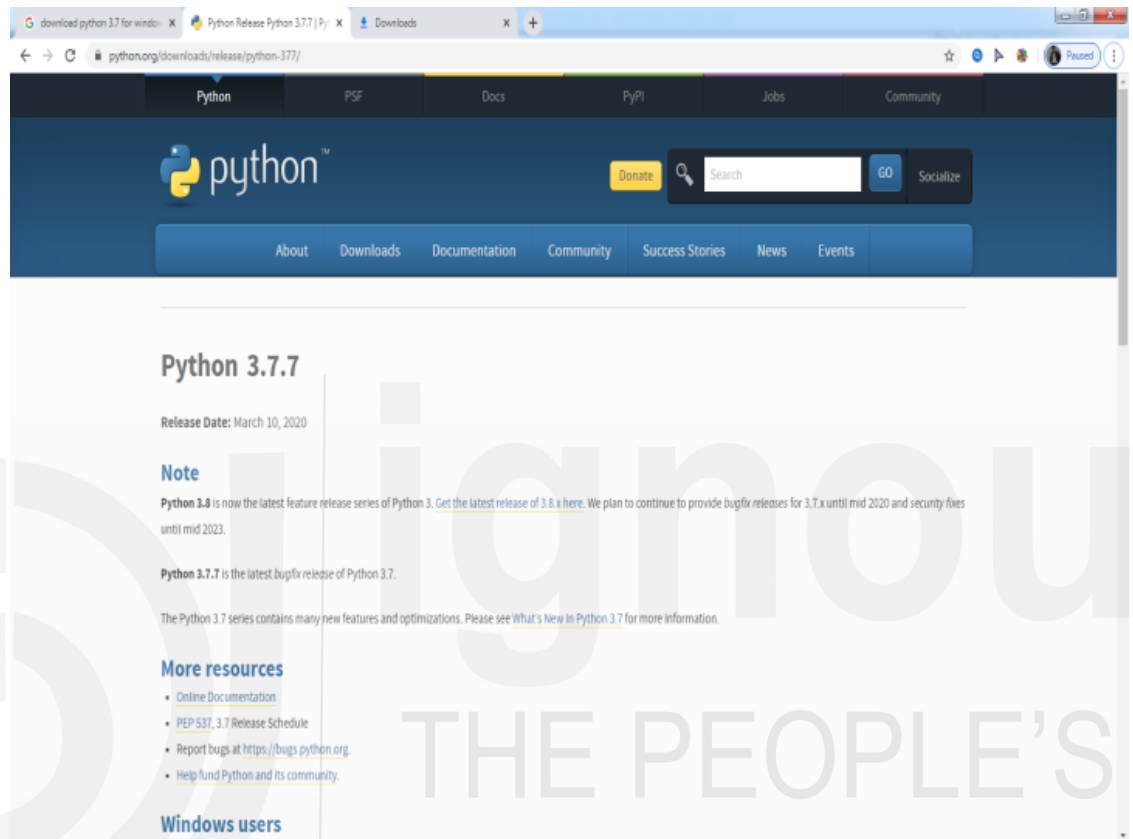


Figure 1: Downloading Python

There is a variety of IDEs for Python, like Jupyter Notebook, Spyder, VS Code, etc, and all are available at Anaconda (a free and open-source distribution of the Python and R programming languages for scientific computing (data science, machine learning applications, large-scale data processing, predictive analytics, etc.)). To start with Anaconda, just perform the following steps.

1. Just browse <https://www.anaconda.com/> and perform the following steps:
 - a) Click the Download button on the webpage of <https://www.anaconda.com/>
 - b) The distribution section <https://www.anaconda.com/distribution/> will open,
 - c) Click the download option given on this page <https://www.anaconda.com/distribution/>
 - d) <https://www.anaconda.com/distribution/#download-section> will open; the 64-bit and 32-bit graphical installer options for Python 3.x (currently 3.7) and 2.x (currently 2.7) are available under Anaconda 2020.02 for Windows installer.

- e) It is recommended to download 64bit version of Python 3.x (currently 3.7),
- f) Anaconda3-2020.02-Windows-x86_64.exe will be downloaded.
- g) Now, just run the setup of this Anaconda3-2020.02-Windows-x86_64.exe , and click next-next, till the installation is completed.
- h) Finally, you will find Anaconda Navigators shortcut on your desktop, click on it and you can start working with any IDE be it Jupyter Notebook, Spyder, VS Code etc., even you can work with R programming.

Important: Before working with IDEs, you need to understand how to work with them and which one is suitable. The following are observations, currently:

1. When you start Jupyter Notebook on Windows (by clicking on the Jupyter section given in Anaconda Navigator), a browser will open in Internet Explorer; many times, Jupyter Notebook won't work here. Just copy the URL from Internet Explorer and paste it into Google Chrome; you will find that Jupyter Notebook starts working. Other details regarding writing, saving, and execution of the program will be discussed later.
2. Program execution in Jupyter is line by line, and in VS Code and Spyder it goes sideways; even errors can also be seen sideways. Anyway all are good to work with Python.

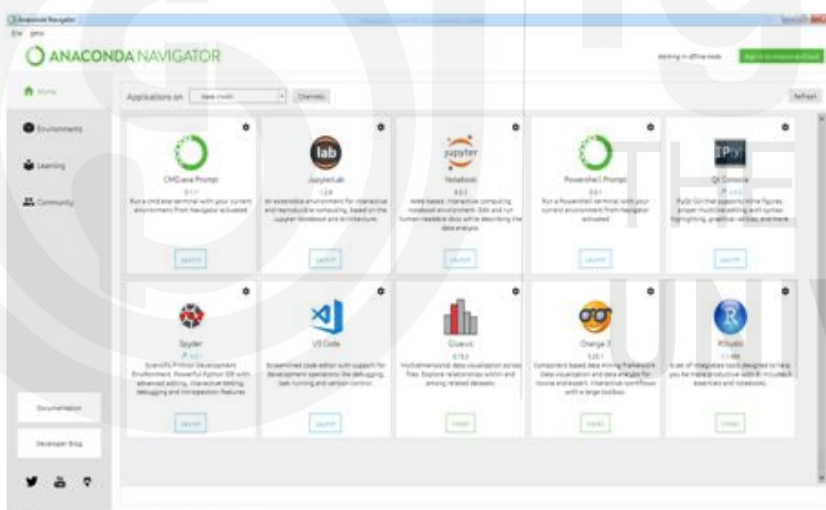


Figure 2: ANACONDA NAVIGATOR

1. Many a time, the learners may not be equipped with systems having the latest hardware configurations, as desired for the installation of Python, or there might be compatibility issues with the operating system, or maybe due to any reason you are not able to install and start your work with Python. Under such circumstances, the solution is Google Colab Notebook (<https://colab.research.google.com/notebooks/welcome.ipynb>). Use this and just log in with your Gmail account and start your work on Jupyter Notebook, as simple as that.

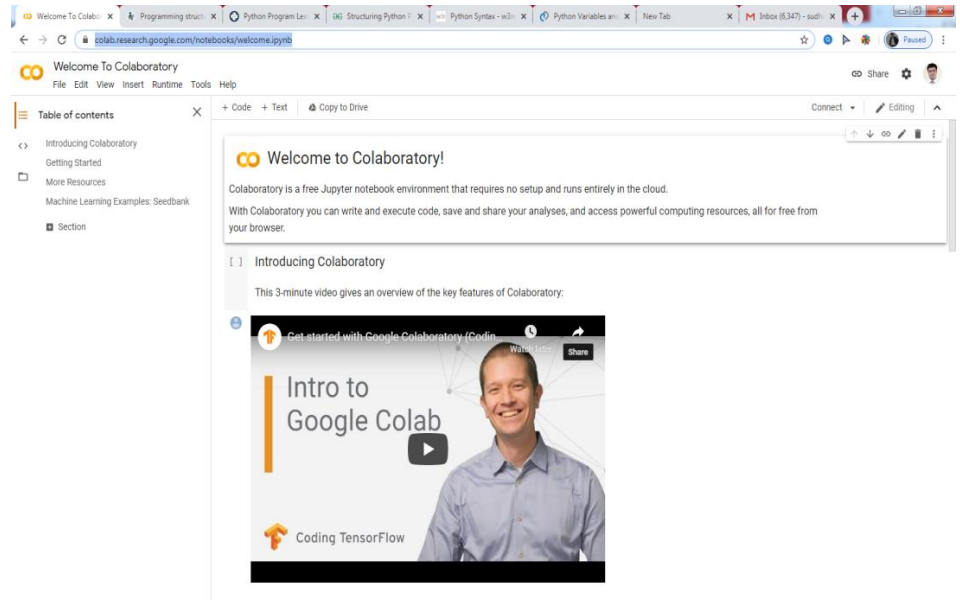


Figure 3: Google Colab

Also, you can use PyCharm, a dedicated Integrated Development Environment (IDE) for Python development built by JetBrains. It is widely considered one of the most powerful tools for Python developers. You can download it from <https://www.jetbrains.com/pycharm/download/?section=windows>.

You will see that many examples in this course material are written using PyCharm.



Figure 4: PyCharm IDE

1.6 PROGRAM STRUCTURE IN PYTHON

This section relates to the discussion over the programming structure of Python programming, i.e. the way you divide a program into your source files and mix parts from various libraries or modules or packages into a single program, i.e. directly or indirectly, a single program includes multiple files from different modules. Modules are, in fact, library tools, which are implemented to make a collection of top-level files; i.e., the high-level files may use tools defined in modules, which may use files defined in other modules. Coming to our point, in python a file takes a module to get access to the tools it defines, and also to the tools defined in other modules included in the program. In Python, a high-level file has important control of your program, where from you can start your application. Just refer to Figure 5 given below.

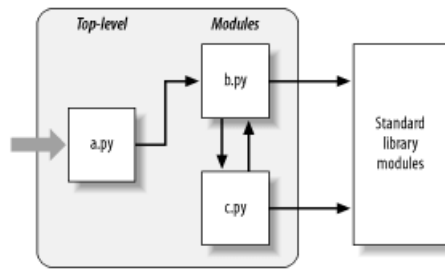


Figure 5 : Structure Python Program

To understand the structure of a Python program, you have to note that there exist three files: **a.py**, **b.py** and **c.py**. The file model **a.py** is chosen for a high-level file; it is known as a simple text file of statements. Files **b.py** and **c.py** are modules. They are also text files of statements, but they are generally not started directly; they are invoked by **a.py**, i.e., the high-level file.

This is just a general discussion over the structure of a Python program, but a Python program also includes all the components as they are in any other object-oriented programming language, such as **data types**, **conditional statements**, **looping constructs**, functions, file handling, Classes, Exception handling, Libraries, Modules, packages etc. We will discuss a few here, and cover the rest in the subsequent units of this course.

1.7 RUNNING THE FIRST PROGRAM

Click File, select New Workbook, and a new Jupyter notebook will open in Google Colab. Now you may start your work

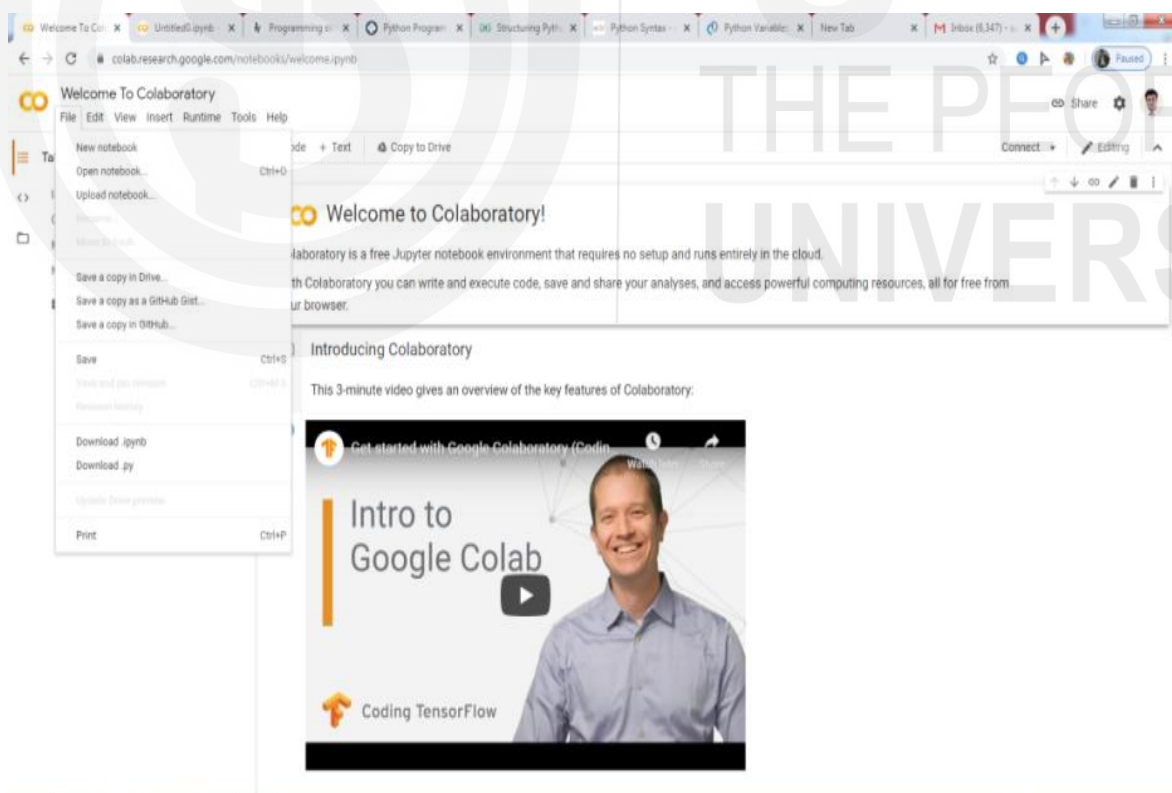
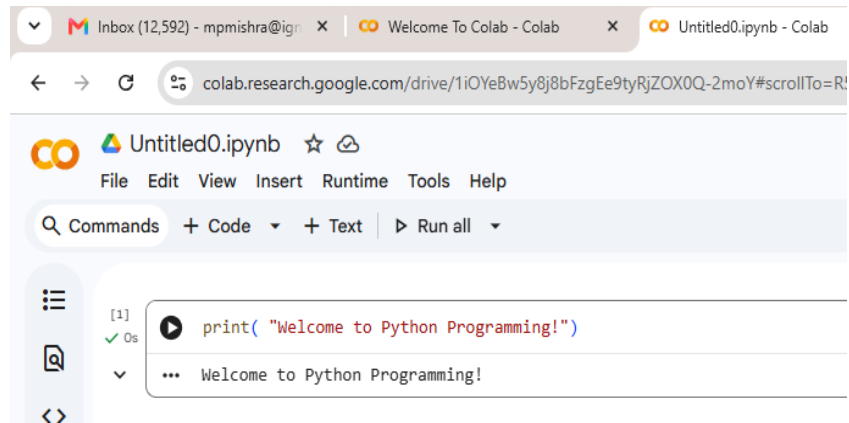


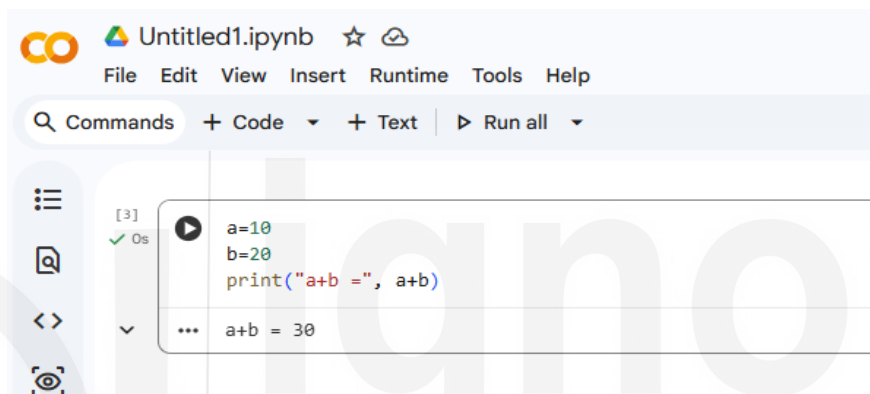
Figure 6: Google Colab

As here, I wrote my first program (`print("Welcome to Python Programming!")`), and executed it by simply pressing the **play button**. You may see, just before the print command I wrote, the output appear beneath the statement `print("Welcome to Python Programming !")`.

First Python program:



Another Python program (to add two numbers):



Check Your Progress -2

1. Compare Framework, Library, Package and Modules in Python.

.....

.....

.....

.....

2. Discuss the utility of the following in Python programming:

- NumPy
- Pandas
- Matplotlib
- Kivy

.....

.....

.....

.....

3. Write a Python program in Google Colab which takes two numbers as input and displays their sum.

.....

.....

.....

.....

1.8 SUMMARY

Guido van Rossum created Python in 1991 to prioritise clean code readability and rapid developer productivity. This unit discussed the history, need for Python and its area of applications. This unit covers not only Python's history, but also its application areas and the tools relevant to exploring them. As described in the unit, Python has an easy-to-learn syntax, dynamic typing, and a vast standard library that powers fields like AI, data science, and web development. For building and distributing cross-platform applications, Python relies on robust packages like Tkinter, PyQt, and Kivy . To start Python programming, you need to install the interpreter from python.org and choose between real-time testing in the interactive shell or writing standalone .py script files. Many platforms/IDEs, like Google Colab and PyCharm, are available for Python programming.

1.9 ANSWER TO CHECK YOUR PROGRESS

Answer to Check Your Progress -1

1.

The following are some key comparisons of Python and C:

- i. Python is a high-level, interpreted language focused on developer productivity and readable syntax, whereas C is a low-level, compiled language built for high execution speed and direct hardware control.
- ii. While Python features dynamic typing, automatic garbage collection, and a massive standard library, C requires static variable declarations and manual memory allocation
- iii. Being interpreter-based, debugging in Python occurs line-by-line during runtime, while C compiles the entire source code at once to catch errors before execution.
- iv. Python is best suited for AI, data science, and rapid prototyping, while C remains the basis for operating systems, embedded hardware, and performance-critical software.

A developer should choose Python when developer speed, ease of readability, rapid prototyping, and vast library support (like AI or web frameworks) are the priority. On the other hand, developers should choose C when execution speed, tight hardware control, low memory overhead, and direct system access (like kernels or firmware) are the key considerations.

2.

- CPython: The standard, C-based reference implementation that compiles Python code into interpreted bytecode. It is best suited for General application development, data science, ML/AI, and script writing.
- Jython: Implemented in Java to compile Python code into Java bytecode for seamless execution on the JVM. Suitable for Legacy enterprise Java systems, JVM scripting, and cross-platform Java app integrations.
- IronPython: Built for Microsoft's .NET Common Language Runtime (CLR), providing native interop with C# and .NET. It can be used for Windows desktop apps, embedded scripting in .NET software, and C# ecosystem tools.

- PyPy: A high-performance variant using a Just-In-Time (JIT) compiler to execute Python significantly faster. It is better in cases of CPU-bound tasks, developing web servers, and algorithm execution.

3.

Some areas of application of Python programming and Python Library/Frameworks used in their development:

- Artificial Intelligence & Machine Learning: Building neural networks, predictive models, and data analytics pipelines (uses Pandas library, and PyTorch and TensorFlow are frameworks).
- Web & API Development: Designing/developing scalable backend applications and web services using frameworks like Django, Flask, and FastAPI.
- Automation & Web Scraping: Automating tedious system tasks, routine file management, and web data extraction (using Selenium, an automation framework/tool suite, and the BeautifulSoup library).
- IoT & Embedded Systems: Scripting for microcontrollers and single-board hardware like Raspberry Pi using MicroPython.

Answer to Check Your Progress -2

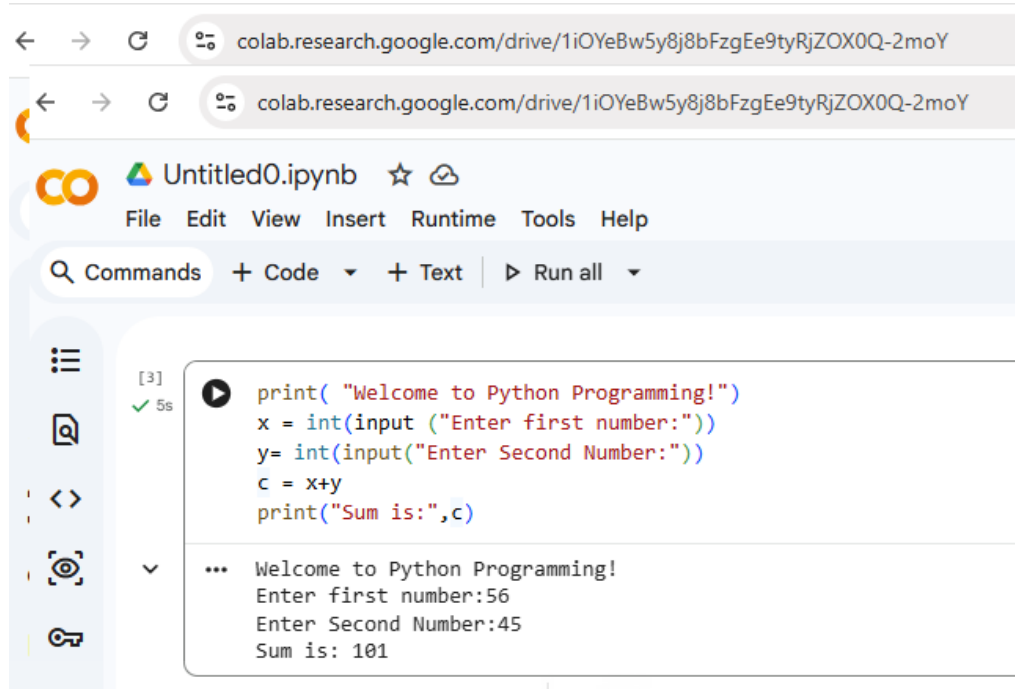
1.

- Module: A single .py file containing reusable code, functions, classes, or variables (math.py, datetime, and os etc.).
- Package: A folder/directory containing multiple related modules along with an __init__.py file (urllib,).
- Library: A broad collection of packages and modules offering specific utilities where your code calls the tools (Pandas, NumPy etc).
- Framework: A complete application skeleton that dictates structure and calls your code via Inversion of Control (Django, Flask etc.).

2.

- **NumPy:** Handles fast math calculations and large grids of numbers. It acts as the main math engine behind almost all Python data science tools.
- **Pandas:** Works with data in tables (just like Excel spreadsheets). It makes it super easy to open, filter, clean, and organise data from files like CSVs
- **Matplotlib:** Turns data into visual charts and graphs (like bar graphs and line plots). It gives you complete control over every detail of how your graph looks.
- **Kivy:** Used to build touch-friendly app screens (GUI) that can run on both computers and mobile phones (Android and iOS).

3.



The screenshot shows a Google Colab notebook titled 'Untitled0.ipynb'. The code cell contains the following Python code:

```
print( "Welcome to Python Programming!")
x = int(input ("Enter first number:"))
y= int(input("Enter Second Number:"))
c = x+y
print("Sum is:",c)
```

The output of the code is displayed below the code cell:

```
... Welcome to Python Programming!
Enter first number:56
Enter Second Number:45
Sum is: 101
```

1.10 REFERENCES/FURTHER READINGS

1. Yashavant Kanetkar and Aditya Kanetkar, "Let Us Python", 4th Edition, BPB Publications, New Delhi , 2022
2. Paul Deitel and Harvey Deitel , "Python For Programmers" , 1st Edition, Pearson India Education , Chennai, 2023
3. Sheetal Taneja and Naveen Kumar, "Python Programming a Modular Approach" , 1st Edition, Pearson India Education , Chennai , 2022
4. S. Shridhar, J. Indumathi and V.M. Hariharan , "Python Programming" , 1st Edition, Pearson India Education , Chennai, 2024
5. S. Suma Christal Mary , R. Josphine Leela, N. Senthamilarasi and Y. Chrisbin Jeeva , "Problem Solving and Python Programming", 1st Edition, Ane Books , New Delhi, 2018